Evaluating training data variability effect on GNN generalization in shortest-path approximations

Dominic Willis[1], Mitchell Black[2], Jesse He[2]

[1] Scripps Ranch High School, San Diego, California
[2] University of California San Diego, San Diego, California

KEYWORDS: *GNN, Generalization, Graphs, Shortest-path*

OVERVIEW: This article explores the effect of varying various aspects of training data in training a neural network to generalize a solution to the shortest-path distance problem. It explores methods that affect the degree of generalization and those that cause shortcut learning in graph neural networks.

SUMMARY

Shortest-path distances are important values in the application of graph theory to real-world scenarios. There exist traditional algorithms to calculate the shortest-path distances, but these are limited in applications involving complex node and edge features or other variable context, something which limits applicability. In this work, we used graph neural networks in approximating shortest-path distances as a precursor to more complex applications. Specifically, we sought to determine the effect of limiting or expanding the amount and variability of training data in supporting generalization of a GNN in predicting shortest-path distances. We hypothesized that, in a GNN predicting shortest-path distances, increasing training data variability would encourage the model to learn a solution that generalizes across graph sizes. We found that when we increased both the amount and variability of the training data by including more graphs in training with different node counts per graph, the model developed a more generalized solution for graphs smaller than the largest graphs trained on and was more likely to generalize past the size of its largest training graphs. Conversely, when we limited the amount of data by only training a model on nodes at a specific radius from the source node, the models generally learned to guess a solution that approximately matched training data but did not generalize a solution that worked well for more variable data. From our experiments, we observed that message-passing GNNs are susceptible to shortcut learning and benefit from more variable training data like other neural networks.

INTRODUCTION

Many real-world systems can be represented by graphs of nodes and edges. This representation can be applied to many systems: from the physical, such as roadways with nodes being intersections and links being roads, to the abstract, like representations of human interconnection or data transfer via the internet. Indeed, any system that involves distinct parts that are interconnected can be described by a node-and-edge graph (1).

In these systems, it can be helpful to minimize distance. In a road system, for example, it is useful to know which path from start to destination is the shortest (2). However, this also requires a modification to the original structure of the graph; different roads, and thus different edges, can have different distances. A weighted-edge graph fulfills this need. Weights can account for distance, as in the road example, but are not limited to that (3). For example, if someone were to ship something a long distance, that might rely on several package carriers with variable shipping costs and varying ability to hand the package off to the next carrier; in this case, weighted edges could represent the cost of each carrier.

A traditional algorithm for shortest-path distances, the Bellman-Ford algorithm, runs iteratively out from a source node, applying a minimization function, potentially with considerations of edge weights, to find the shortest distance for nodes based on their neighbors' current estimate of the distance to the source and edge weights (2). Over several iterations, this has the effect of considering information from increasingly separated nodes (4). The Bellman-Ford algorithm, however, involves no inferencing with complex features (2), and is thus limited in applications to more complex systems compared to neural networks (5).

To understand graphs with more versatility, deep learning models have been developed, relying on an understanding of the importance of iteratively sharing information to inference on graphs (6). These deep learning models are called graph neural networks (GNN). The deep learning model uses a method called message-passing (7): each node is initialized with some feature to begin the message-passing—in our case, a value far from the range of expected values—and each layer of the network updates this feature until all layers have been used and a prediction is created. The network uses the node and its neighbors in performing each step of the message passing scheme. During training, these layers can then be optimized towards predicting correct values. Effectively, these deep learning models can apply information from the nodes and from the edges to iteratively make predictions, accumulating data from further nodes with each

Commented [DW1]: https://networksciencebook.com/chapter/1#networks Section 1.2

Commented [DW2]: https://mcube.lab.nycu.edu.tw/~cfung/docs/books/cormen2001algorithms.pdf page 580

Commented [DW3]: https://networksciencebook.com/chapter/2#weighted-networks Section 2.6

Commented [DW4]: https://mcube.lab.nycu.edu.tw/~cfung/docs/books/cormen2001algorithms.pdf page 588

Commented [DW5]: https://dl.icdst.org/pdfs/files3/9ce98c127c79e548ebea18966f526ae9.pdf page 294

Commented [DW6]: https://mcube.lab.nycu.edu.tw/~cfung/docs/books/cormen2001algorithms.pdf page 588

Commented [DW7]: https://arxiv.org/pdf/1901.00596

Commented [DW8]: https://arxiv.org/pdf/1806.01261 4.3, 4.4

Commented [DW9]: https://arxiv.org/pdf/1704.01212 term origin

aggregation. Neural networks can generalize solutions in this way, but they can also be susceptible to shortcut learning, where the model develops a solution that generalizes poorly (8).

We hypothesized that for a GNN predicting shortest-path distances, increasing training data variability would encourage the model to learn a solution that generalizes across graph sizes. Experimentation on this hypothesis revealed that GNNs generalized a solution to the shortest-path problem when provided graphs with varied data, such as varied node counts. Limiting training to the extreme of only training on nodes at a certain radius from the source encouraged shortcut learning, resulting in poor generalization to the problem. Thus, the data and their variability provided to a GNN affect the development of a generalized solution to the shortest-path problem.

RESULTS

Experiment 1: Assessment of the Generalization of Networks Trained on Graphs of Specific Sizes

In this experiment, we trained models on separate collections of graphs, where each collection had graphs with a specific number of nodes. We then tested these separate networks—which were each trained on a separate graph collection—on testing graphs of varying size to assess generalization. Our intent was to assess the ability of these networks trained on data that was limited to graphs of a certain size to generalize a solution across graphs of more varied sizes. We suspected that the graphs would display a positive relationship between test graph size and the GNN testing loss, and a negative relationship between training graph size and GNN testing loss. The results upheld this hypothesis, with a positive correlation between $\log_{10}$ of test loss and testing graph size ($4.645012 \times 10^{-3} \pm 1.5 \times 10^{-10}$; $p<0.025$, two-variable multiple linear regression), and a negative correlation between $\log_{10}$ of test loss and training graph size ($-4.258683 \times 10^{-3} \pm 1.2 \times 10^{-9}$; $p<0.025$, two-variable multiple linear regression). In general, low test loss results were observed for networks being tested on graphs of similar size to their training graphs, poor results were observed for most models when tested on graphs of a size larger than their training graphs, and results for graphs smaller than the training graphs were more varied but with generally better results than with testing graphs larger than the training graphs (**Figure 1**).

Experiment 2: Assessment of the Generalization of Networks Trained on Graphs of Varying Sizes and its Comparison to the Generalization of Networks Trained on Graphs of Specific Sizes

In this experiment, we used a setup similar to the previous experiment, except the training data included all graphs up to the node count, meaning that networks with larger graph size values also had more training data and more variability in that data. We expected that introducing more

variability in the training data would force the model to develop a more generalized solution than the networks trained separately on graphs of the same sizes; a positive correlation would be present between testing graph size and test loss, and a negative correlation would be present between training graph size and test loss, but both would be significantly closer to zero than the previous experiment. The results upheld this hypothesis for both training graph size ($4.645012 \times 10^{-3} \pm 1.5 \times 10^{-10}$ vs $2.186042 \times 10^{-3} \pm 1.7 \times 10^{-10}$; $p<0.025$, separate multiple linear regressions) and testing graph size ($-4.258683 \times 10^{-3} \pm 1.2 \times 10^{-9}$ vs $-2.225597 \times 10^{-3} \pm 1.3 \times 10^{-9}$; $p<0.025$, separate multiple linear regressions) with correlations to $\log_{10}$ of test loss. We also observed some models generalizing well past the maximum node count (**Figure 2**). In addition, experiment 2 showed generally lower test loss results than experiment 1 (2.008 ± 0.049; $p<0.025$, paired-values t-test: $\log_{10}$ of Experiment 1 test loss minus $\log_{10}$ of Experiment 2 test loss). The fractional comparison of the two experiments (experiment 2 test loss divided by experiment 1 test loss) shows improvement, particularly for higher testing graph sizes (**Figure 3**).

Experiment 3: Assessment of the Generalization of Networks Trained Only on Nodes at a Specific Radius

Having observed that the model could generalize, at least partially, even when the training data for each network was limited to being of a certain size, we experimented with the further limitation of only training and testing on nodes of an exact radius (the distance to the source node) on their shortest path. The results indicated that the models likely developed a phenomenon known as shortcut learning, where the limitations on the model's training data led it to converge on a solution that does not generalize to more varied testing data (**Figure 4**). In fact, the results for the network trained only on nodes at zero 'hops' from the source node on their shortest path—the source nodes—indicate that the model likely learned to simply guess zero every time. The similarity between both rows in **Table 1** may result from the usage of mean-square error in calculating test loss error. Individual quadratic regressions for each training node count, with test loss calculated based on testing node hops, showed that test loss increased for node radii further from the training node radii ($R^2 > 0.999$, quadratic term coefficient in [1.078,1.239] for training node radii 0, 1, and 2; quadratic regression). A similar occurrence can be observed for other rows, although that occurrence degrades for networks trained on specific nodes with a larger number of edges involved in their shortest paths. The high error for models with higher number of training number of hops on testing nodes with low number of hops may have resulted from the fact that these networks were trained on graphs which tended to have fewer nodes (this results from the configuration of Erdős-Rényi graphs which limits a large hop amount with more nodes in the

graph), while the testing data includes graphs with much larger node size. As seen in the experiment on the exact number of nodes in training graphs and the effects on testing on graphs with higher node counts, the networks were generally only capable of accurate approximations up to their node count amount.

DISCUSSION

Our experiments demonstrate two main results regarding the effect of modifying training data on a graph neural network's ability to generalize a solution to the shortest-path distance problem.

First, the experiments comparing the result of training each model only on graphs of a specific size and the result of training each model on graphs up to a specific size showed improvement from the latter over the former. This suggests that graph neural network models generalize well up to the node count of the graphs on which they are training when focused on the shortest-path distance problem. Applying more variability in the graphs (different node numbers) may improve generalization, although results are not universal. The results suggest that introducing more variability to the training data made it more likely that low test loss would be observed regardless of testing node size.

In this experiment grouping, limited computational resources served as a bottleneck to training more neural networks, as the amount of time needed to generate larger graphs at a scale reasonable for testing stretched into hours. Training was on a similar time scale. Further, we did not repeat the experiment on differing random number seeds (same graph collections, different initialization of model parameters), which, although less important after several epochs, could still have played a role in the results, including the generalization to larger graphs than the training graphs, which seemed to occur randomly. This is because the random number seed influences the starting parameters of the network, which may influence the final optimized location that the trained model arrives at.

For the second result, the experiment that severely limited the training data that the neural network was allowed to train on showed indicators of shortcut learning. While not indicative of any positive development in the training of neural networks to compute shortest-path distances, this experiment does indicate the potential of graph neural networks using convolutional layers to perform shortcut learning, and thus demonstrates the potential for shortcut learning to negatively impact a trained model’s application when limiting the scope of training data in such a way that

testing data could feasibly have some factor (such as being a different distance from the source node than all the training data was) which was not present in training data.

Once again, we could have incorporated more replicates to ensure consistent results by varying the random number initialization. It is also possible that the regularization constant, alpha, may have conduced these models into shortcut learning by encouraging simplicity (9).

For both experiments, it may also have been beneficial to explicitly explore the individual parameters of each model to observe how it was calculating its values and potentially understand what the ideal model parameters would look like.

These experiments still leave significant questions and opportunities for further exploration. The graphs, for example, were entirely Erdős-Rényi; whether this limits a model's ability to generalize to graphs of a different type, and whether inclusion of other graph types in the training data would enhance generalization, remains an area of future study. Additionally, the network architecture used here was based on the convolutional layer, but other graph learning methods exist; whether one of those architectures would be better suited to the shortest-path distance problem remains an area of future study. These experiments all used explicit edge-weight data and demonstrate that networks can generalize a result when given this data. However, in exploring the applicability of the graph neural network convolutional layer, the replacement of explicit edge weight data with data from which the model is expected to infer the edge weight would mark a significant step towards more versatile real-world applications.

In conjunction, these experiments demonstrate that varying graph data encourages the generalization of a graph neural network based on the convolutional layer architecture under simple conditions. This indicates the potential of message-passing graph neural networks in interpreting graphs to solve and generalize a solution to non-trivial problems that have real-world applications. Given that graph neural networks are based around inferencing, it also suggests the potential for the application of a similar architecture to a shortest-path distance problem with inferred edge weights.

MATERIALS AND METHODS

Processing Interface

All experiments used Google Colaboratory's cloud environment. GPU-based training was performed on GPUs available in Google Colaboratory, including the NVIDIA T4 and the NVIDIA A100. Python's Matplotlib library was used to generate heatmaps (10).

Graphs

The experiments all shared certain goals and setups. The experiments all used Erdős-Rényi graphs from NetworkX (where a specific number of nodes are generated and each edge between any set of two nodes has a specific probability of occurring); all graphs used an edge appearance probability of 0.1, and only graphs that were connected were used (11, 12). These graphs were centered, meaning they had one node selected as the source node. In these graphs, edges were weighted as a way of specifying the length of the path between the two nodes, as opposed to unweighted edges. Specifically, the edge weightings followed a normal distribution around 1 with a standard deviation of 0.1. The graphs were also initialized with descriptors for every node: the true shortest path length based on NetworkX calculations using the weighted edge attributes and a large number (on all nodes but the centered node, which had an initialization of 0) for the purpose of providing data for the network message passing. During experimentation, the number of graphs and the number of nodes inside the graphs were varied, and masks were applied to the graphs to influence the effect of specific nodes during the training process.

Network

The network used in the experiments was a neural network based on the PyTorch neural network module (13). The network class took 1 channel of information (the graph data) and output 1 channel of information (the shortest path guesses). Received data was encoded from 1 channel up to a value referred to as hidden_channels. Across all experiments, the network consisted of 4 layers—acting at the hidden_channels level—made of GINEConv (Graph Isomorphism Network with Edge features Convolution) layers, formally defined below, that contain PyTorch Geometric neural network sequential layers with variations in the number of Linear layers and ReLU activations (13, 14). The information was then decoded with a linear layer going from hidden_channels back to the 1 output channel. The formal definition of the GINEConv layer, where h_Θ is the neural network, $N(i)$ refers to the neighborhood of node i (all the nodes that are connected to i by edges); $e_{j,i}$ is the edge feature vector for the edge between node i and j; x_i is the feature vector for node i (x_i' is the updated version of x_i); and ReLU is a non-linear function that prevents negative values: ReLU(a) = max{a,0} (15).

$$x_i' = h_\Theta\left((1+\epsilon)\cdot x_i + \sum_{j\in N(i)} \mathrm{ReLU}\left(x_j + e_{j,i}\right)\right) \quad \textit{(Equation 1)}$$

Training

The network was trained using Torch's Adam optimizer with a learning rate of 0.0003 and batched (batch size of 64) and shuffled training data (16). In training, the loss function used to generate the gradient for optimization of the network was mean-square error loss, where the differences between the prediction and true shortest path value were individually squared and then averaged. Before using this error to optimize the neural network parameters, a regularization constant, alpha, was applied, where the sum of the model parameters was multiplied by alpha and then added to the error (17). The model was then optimized on the overall error. This addition to the training process has the effect of encouraging simplicity in the network, as it punishes having multiple parameters with large values.

$$\text{error} = \frac{1}{n}\sum\left(\hat{Y}_i - Y_i\right)^2 + \alpha\|\theta\|_1 \quad \textit{(Equation 2)}$$

Hyperparameter Choice

To choose the hyperparameters for the network, we ran several tests beforehand. We tested the tendencies of several different hidden_channels values to overfit on multiple different initialization seeds. We also tested the effect that alpha had on test loss and chose a value of 1.5 from those results. To perform this test of alpha, we performed random checks, pseudo-gradient descent, and Bayesian optimization (18). Throughout the experiments, we used a hidden_channels of 4 and an alpha value of 1.5.

REFERENCES

1. Barabási, Albert-László and Márton Pósfai. "Introduction." *Network Science*, Cambridge University Press, 2016, https://networksciencebook.com/chapter/1. Accessed 31 Jan. 2026.
2. Cormen, Thomas H., et al. "Single-Source Shortest Paths." *Introduction to Algorithms*, 2nd ed., MIT Press and McGraw-Hill, 2001, pp. 580-619. Accessed online.
3. Barabási, Albert-László and Márton Pósfai. "Graph Theory." *Network Science*, Cambridge University Press, 2016, https://networksciencebook.com/chapter/2. Accessed 31 Jan. 2026.
4. Kleinberg, Jon and Éva Tardos. "Dynamic Programming." *Algorithm Design*, 1st ed., Pearson Education, Inc., 2005, pp. 251-336. Accessed online.
5. Wu, Zonghan, et al. "A Comprehensive Survey on Graph Neural Networks." *arXiv*, 4 Dec. 2019. https://arxiv.org/abs/1901.00596
6. Battaglia, Peter W., et al. "Relational inductive biases, deep learning, and graph networks." *arXiv,* 17 Oct. 2018. https://arxiv.org/abs/1806.01261

7. Gilmer, Justin, et al. “Neural Message Passing for Quantum Chemistry.” *arXiv,* 12 Jun. 2017. https://arxiv.org/abs/1704.01212
8. Geirhos, Robert, et al. “Shortcut Learning in Deep Neural Networks.” *Nature Machine Intelligence*, vol. 2, no. 2, 10 Nov. 2020, pp. 665-673. https://doi.org/10.1038/s42256-020-00257-z
9. Shah, Harshay, et al. “The Pitfalls of Simplicity Bias in Neural Networks.” *arXiv*, 28 Oct. 2020. https://arxiv.org/abs/2006.07710
10. Hunter, John D. “Matplotlib: A 2D Graphics Environment” *Computing in Science & Engineering*, vol. 9, no. 3, 30 Jun. 2007, pp. 90-95. https://doi.org/10.1109/MCSE.2007.55
11. Erdös, P., A. Rényi. “On the Evolution of random Graphs.” *Publications of the Institute of Mathematics of the Hungarian Academy of Sciences*, 1960. Accessed online.
12. Hagberg, Aric A., et al. “Exploring Network Structure, Dynamics, and Function using NetworkX.” *Proceedings of the 7th Python Conference*, 2008, pp. 11-16. https://aric.hagberg.org/papers/hagberg-2008-exploring.pdf
13. Paszke, Adam, et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” *Proceedings of the 33rd Conference on Neural Information Processing Systems*, 3 Dec. 2019. https://arxiv.org/abs/1912.01703
14. Fey, Matthias, Jan Eric Lenssen. “Fast Graph Representation Learning with PyTorch Geometric.” *arXiv,* 25 Apr. 2019. https://arxiv.org/abs/1903.02428
15. Hu, Weihua, et al. “Strategies for Pre-training Graph Neural Networks.” *arXiv*, 18 Feb. 2020. https://arxiv.org/abs/1905.12265
16. Kingma, Diederik P., Jimmy Ba. “Adam: A Method for Stochastic Optimization.” *arXiv*, 30 Jan. 2017. https://arxiv.org/abs/1412.6980
17. Goodfellow, Ian, et al. “Regularization for Deep Learning.” *Deep Learning,* MIT Press, 2016, pp. 224-270. https://www.deeplearningbook.org/contents/regularization.html
18. Head, Tim, et al. “scikit-optimize.” *Zenodo*. https://doi.org/10.5281/zenodo.4014775. Accessed 5 Feb. 2026.

Figures and Figure Titles/Captions

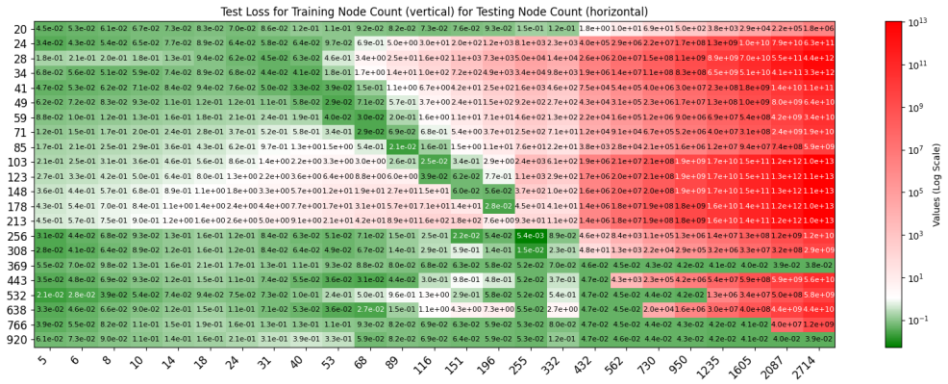

Figure 1. Test loss for testing node count by exact training node count. Heatmap showing test loss (n=1). Networks were trained on graphs of a specific node count and then tested on a range of graphs with specific node counts.

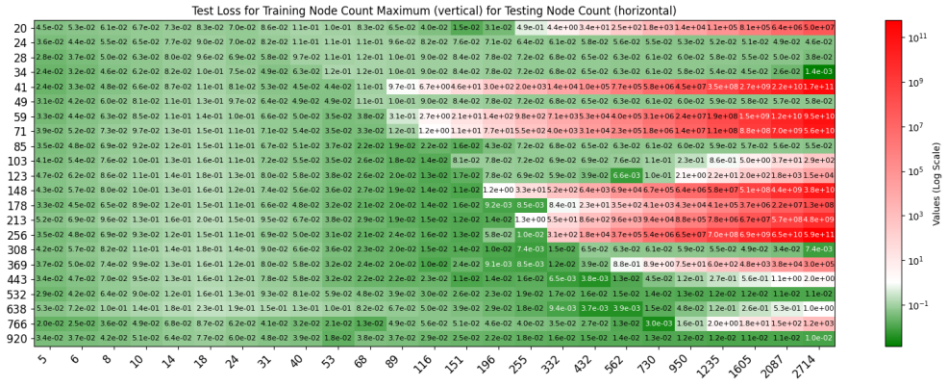

Figure 2. Test loss for testing node count by maximum training node count. Heatmap showing test loss (n=1). Networks were trained on graphs with node counts of or smaller than the specified node count and then tested on a range of graphs with specific node counts.

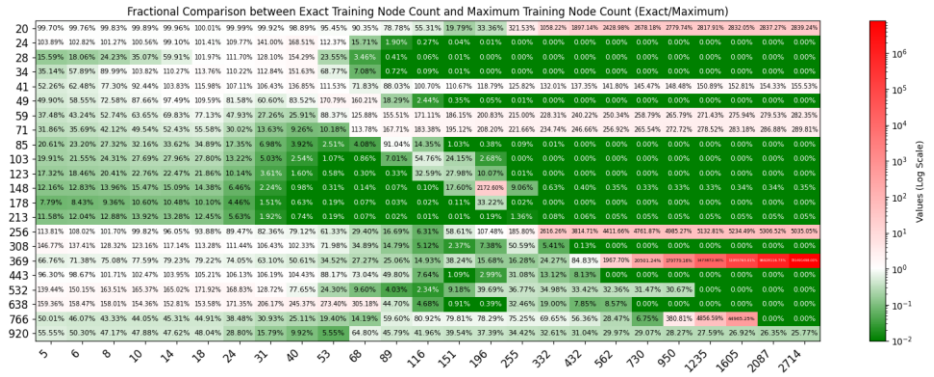

Figure 3. Fractional comparison between networks trained on exact node count graphs and networks trained on maximum node count graphs. Heatmap showing variation (green indicates that the maximum node count training method showed improvement over the exact node count training method) (n=1). The value for each box results from the division of the test loss from the maximum node count arrangement by the test loss from the exact node count arrangement.

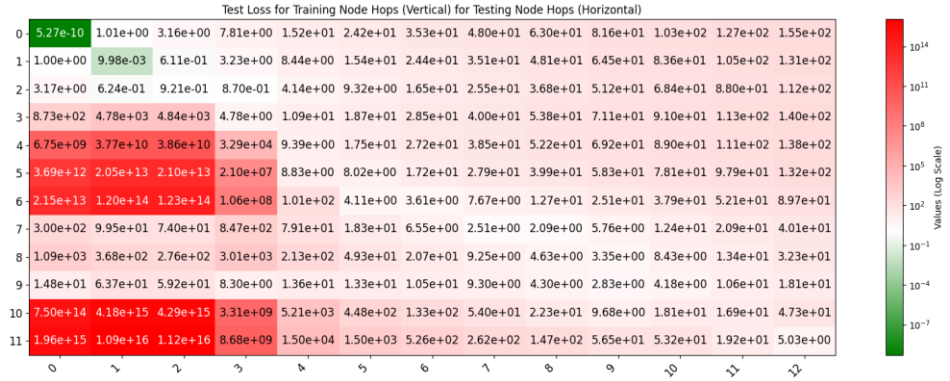

Figure 4. Test loss for testing node distance by training node distance. Heatmap showing test loss (n=1). Networks were trained on nodes that had a true shortest path distance of a specific amount through masking and were tested on a range of nodes that had true shortest-path distances of specific amounts.

Tables with Titles/Captions

n	0	1	2	3	4	5	6	7	8	9	10	11	12
n^2	0	1	4	9	16	25	36	49	64	81	100	121	144
Test Loss	0.00	1.01	3.16	7.81	15.2	24.2	35.3	48.0	63.0	81.6	103	127	155

Table 1. Test loss compared to the number of edges involved in the shortest path for the testing nodes. A table demonstrating the similarity between test loss and the square of the number of edges involved in the shortest path (n) for specific nodes. These results are from a network trained only on source nodes (0 edges involved in the shortest path) and are indicative of shortcut learning.

Appendix

GitHub Repository: https://github.com/Iver727/shortest-path-gnn-experiment